

2026-04-30

Evolving mmints from Citizen Science to Cyborg Science

Mackson Ncube
Mightymetrika, LLC

This template was last revised on 2026-03-24.

Executive Summary

The Problem

Applied researchers frequently justify their statistical choices by citing simulation results from methodological papers, even when the original simulation parameters do not match their specific study context. While the rigorous solution is to re-test the simulation under new parameters, this is often prohibitive due to time constraints, unreproducible code, or limited technical bandwidth. Porting these methods into web-based Shiny applications democratizes exploration, but current “citizen science” platforms rely entirely on human curiosity. This may leave blind spots in the parameter space and create a disconnect between lightweight web exploration and formal, research-grade publication.

The Solution

We propose evolving the `mmints` R package from a citizen science platform into a “cyborg science” collaborative environment. By integrating Large Language Models (LLMs) directly into the simulation pipeline via native Shiny modules, we will bridge the gap between web-based parameter exploration and full-scale server execution.

Deliverables & Methods

This project will deliver a CRAN-ready update to the `mmints` package featuring two primary Shiny modules:

- **Intelligent Parameter Exploration (LLM Credit Donation):** A module allowing users to donate API credits so an LLM can ingest research context, analyze the open database, and intelligently search the parameter space to discover and save interesting edge cases.
- **AI Reporting & Code Generation:** A conversational assistant that analyzes the aggregated database to draft comprehensive reports and output the exact, executable R code needed to run a full-scale (10,000+ iteration) simulation on a server.

Expected Outcomes

We will provide the R community with standardized, open-source infrastructure to seamlessly augment human researchers with AI. This will transform passive simulation databases into active engines for discovery, directly facilitating the generation of robust scientific papers from web-based inputs.

Timeline & Budget

The project will be completed over a six-month timeline. The requested budget is **\$15,000 USD**. Because the foundational database and package infrastructure are already deployed, this budget is highly optimized, strictly covering the labor required to build, test, and deploy the native R API wrappers, context injection pipelines, and Shiny modules.

Signatories

Project team

Mackson Ncube (Principal Investigator)

As a professional statistician and data scientist, Mackson is the founder of mightymetrika, a think tank dedicated to developing statistical tools for small sample size research. The mightymetrika also specializes in developing citizen science applications to crowd source the investigation of statistical methods. He is the developer of the `mmints` R package.

- Profiles: [ResearchGate](#) | [UpWork](#)

The Problem

The Simulation Context Mismatch

In statistical methods research, a common anti-pattern persists: applied researchers frequently cite simulation results from original methods papers to justify a statistical choice, even when the original simulation parameters do not match their new study's specific context.

The scientifically rigorous response is to utilize a REPLEXT (replication and extension) function—a tool designed explicitly to re-test published simulation results under study-specific parameters. Unfortunately, this is rarely done in practice. Time constraints, unreproducible code, and a lack of technical bandwidth within applied research teams prevent these vital extensions from occurring.

The Limits of Human-Driven Citizen Science

Currently, a vast amount of statistical knowledge lives exclusively in the heads of individual methodologists who run informal, unpublished simulations to satisfy their own curiosity about edge cases. Without a centralized community space, the broader R ecosystem misses out on this collective intelligence.

We have laid the groundwork to solve this by porting methods into web-based Shiny citizen science apps. Live [mightymetrika citizen science applications](#) (e.g., `npreplext032`) are currently deployed, allowing users to test smaller iterations of a method and save the results to an open-access database. However, relying solely on human engagement limits the scale of this exploration.

The Bottleneck: Evolving to Cyborg Science

While creating an open database of citizen-science simulations is a massive leap forward, two bottlenecks remain: efficiently exploring a vast parameter space, and synthesizing crowd-sourced data into formal, research-grade extensions.

This grant aims to build infrastructure that seamlessly augments human researchers. We propose bridging the gap between web-based citizen science and formal scientific publication by evolving `mmints` into a human-AI collaborative environment through two focused modules:

1. **Intelligent Parameter Exploration:** Relying solely on human curiosity leaves blind spots in the parameter space. We propose a module where users can donate API credits, allowing an AI to ingest the original methods paper, evaluate the current database, and follow user prompts to intelligently select and run new simulation parameters. Functioning similarly to an AI-driven Markov process, it will perform a correlated search to discover the most scientifically interesting edge cases.
2. **AI Reporting Module:** Synthesizing the database into a research-grade extension remains a challenge. We will build a chat module where the AI analyzes the open database to draft a comprehensive report based on user queries. Crucially, the AI will output the exact R functions needed to run the optimal full-scale simulation, keeping the human firmly in the loop to execute the final research-grade simulation on their own server.

The Proposal

Overview

We propose updating the `mmints` R package to provide specialized Shiny modules that transform web-based statistical citizen science applications into “cyborg science” environments. By integrating Large Language Models (LLMs) directly into the simulation pipeline, we will bridge the gap between lightweight parameter exploration and research-grade REPLEXT execution.

Proof of Concept and Existing Infrastructure

This project minimizes delivery risk by building upon a robust, existing foundation.

- **Deployed Citizen Science Apps:** The `mmints` framework is actively deployed in production. We currently maintain multiple open-source REPLEXT Shiny applications (e.g., `npbreplext`, `mmirreplext`, `regls`), demonstrating that the community architecture for saving and aggregating simulation parameters is functional.
- **The `shinypup` AI Proof-of-Concept:** In September 2024, we developed an exploratory AI bot called `shinypup`. Its specific goal was to act as an “AI live blogger” that observed and reported on a statistical simulation using external browser automation (Puppeteer navigating a Shiny app) ([Live Demo Recording](#)).

The Evolution to Native Integration

The objective of the `mmints` update is to help the research community collaborate directly with AI to produce new, research-grade knowledge via replication and extension (REPLEXT) simulations. To achieve this, we will build stable, native R modules that establish direct communication between the AI and the Shiny application’s underlying ui and server architecture. This shifts the AI from an external observer “clicking” a browser to an integrated, internal collaborator.

Minimum Viable Product

The MVP will be a CRAN-ready update to the `mmints` package containing two fully functional Shiny modules:

1. **Intelligent Parameter Exploration (LLM Credit Donation):** A module allowing users to input an LLM API key to drive automated exploration.
2. **AI Reporting & Code Generation:** A conversational module synthesizing database results into a downloadable report containing executable R code for server-side REPLEXTs.

App builders will be able to plug these modules into their existing simulation apps with minimal friction.

Architecture

The architecture centers on creating modular, extensible Shiny UI/Server components:

- **LLM API Interfacing:** Secure, native R API wrappers to connect the Shiny environment to major LLM providers.
- **Context Injection Pipeline:** Mechanisms for app builders to hardcode foundational research papers into the system prompt, and for end-users to upload supplementary PDFs.
- **Database Optimization:** Refined data-handling logic ensuring the AI can rapidly query and summarize aggregated citizen science results.
- **Safety & Tracking:** Implementation of prompt sanitization and tracking for user-donated LLM queries and data contributions.

Assumptions

- Researchers are willing to provide their own LLM API keys or donate credits to drive the exploration process.
- Modern LLMs are capable of consistently outputting syntactically correct R code for server-side REPLEXT functions based on package documentation.
- LLM context windows are sufficient to ingest aggregated database summaries alongside multiple academic papers.

External Dependencies

- Existing `mmints` UI/Auth dependencies: `shiny`, `DT`, `shinyauthr`, `sodium`.
- Database management packages: `RPostgres`, `pool`.
- API interaction packages: `httr2`, `jsonlite`.
- External LLM APIs (e.g., OpenAI, Anthropic) accessed via user-provided keys.

Project Plan

Start-up Phase

Because the foundational `mmints` package is complete, the start-up phase will be highly streamlined.

- **Collaboration & Tracking:** All development will be tracked publicly via the Mightymetrika GitHub.

- **Licensing:** The new modules will be released under the open-source MIT license, or the most permissive open-source license allowable by newly introduced project dependencies.
- **Reporting Framework:** We will establish a bi-monthly internal review schedule to ensure the API integrations and database optimizations meet performance benchmarks before public beta testing.

Technical Delivery

The project will be executed over a six-month timeline, commencing upon grant acceptance (anticipated July 2026):

- **Months 1–2 (July - August 2026): Core API & Context Infrastructure**
 - Develop secure, native R API wrappers (utilizing `httr2` and `jsonlite`) for LLM interaction.
 - Build the context injection pipeline for foundational papers and user-uploaded PDFs.
- **Months 3–4 (September - October 2026): Intelligent Parameter Exploration Module**
 - Develop the Shiny module for user LLM credit donation.
 - Integrate the AI’s parameter generation logic with robust database handling (`pool` and `RPostgres`). This ensures the AI can efficiently query existing results to find unmapped edge cases, and that newly generated simulation results are safely written back to the database without risking connection overload.
- **Month 5 (November 2026): AI Reporting Module**
 - Build the conversational UI for the reporting assistant.
 - Finalize prompt engineering to ensure the AI synthesizes database queries into a downloadable Markdown report containing executable server-side R code.
- **Month 6 (December 2026): Documentation, Testing, & Deployment**
 - Comprehensive testing of prompt sanitization and database load handling.
 - Write package vignettes demonstrating the implementation of the modules.
 - Submit the updated `mmints` package to CRAN.

Publicity & Community Engagement

- **Open Source:** All code hosted publicly on GitHub; final package deployed to CRAN.
- **Blog:** A comprehensive post detailing the transition from citizen science to “cyborg science” using `mmints`.
- **Conferences:** We intend to submit a talk to useR! or similar R conferences to demonstrate the deployed modules.

Budget & Funding Plan

The total requested budget for this project is **\$15,000 USD**.

Because the underlying `mmints` package and database architecture are established, this budget is highly efficient. Funds will be allocated entirely to labor costs for core development, testing, and CRAN deployment.

In accordance with R Consortium ISC guidelines, the funding will be structured as follows:

- **Initial Payment (50% - \$7,500):** Paid upon signing the contract. Covers the start-up phase, core API development, context infrastructure, and the initial build of the intelligent parameter exploration module.
- **Final Payment (50% - \$7,500):** Paid upon project completion. Covers the finalization of the AI reporting module, safety testing, documentation, and successful CRAN submission.

Success

Definition of Done

The project will be considered successfully completed when:

1. **Module Integration:** Both the Intelligent Parameter Exploration and AI Reporting modules are fully integrated into the `mmints` R package as robust, native Shiny modules.
2. **CRAN Acceptance:** The updated `mmints` package successfully passes all checks and is published on CRAN.
3. **Documentation:** Comprehensive vignettes are published, demonstrating how R developers can integrate these modules into their own Shiny-based statistical simulation apps.
4. **Knowledge Sharing:** A project summary and tutorial are published online.

Measuring Success

We will track the impact of the project through both technical and community-driven markers:

- **Technical Markers (During Development):**
 - Error-free API execution of LLM prompts from within the native R environment.
 - Successful translation of LLM outputs into syntactically correct R code that executes on the server without breaking the `pool` database connection.
- **Community Markers (Post-Deployment):**
 - **Adoption:** The number of unique Shiny applications built by the community that implement the new `mmints` AI modules.
 - **Exploration Volume:** The number of AI-driven parameter explorations logged in the databases of deployed `mmints` apps.
 - **Engagement:** Standard open-source metrics (GitHub stars, forks, issues, and pull requests).

Future Work

Establishing this infrastructure opens several highly practical avenues for future development within the R ecosystem:

1. **Containerized Deployment Infrastructure:** Currently, deploying a robust REPLEXT Shiny application requires significant backend overhead. Future work will focus on building standardized deployment templates or Docker configurations within `mmints` to allow researchers to spin up these complex, AI-integrated environments with minimal friction.
2. **Agentic Generation of REPLEXT Ecosystems:** Mightymetrika’s *Random Playables* is a live platform designed to “turn mathematical ponderings into citizen science games.” It successfully utilizes an AI chatbot to dynamically build TypeScript/React applications and

connect them to the platform’s database. Leveraging the core principles of this AI-driven generation, future iterations of `mmints` will strive to develop an AI agent tailored for the R ecosystem. This agent will be capable of ingesting a methodological paper and automatically generating the corresponding REPLEXT functions and a fully structured REPLEXT Shiny application.

3. **Towards Autonomous AI Methodologists:** By establishing the foundational client-server communication modules in this grant, we create the stepping stones to move from a human-in-the-loop “cyborg scientist” toward fully autonomous AI research agents. Future development could allow these agents to autonomously monitor methodological literature, run necessary parameter extensions, and report optimized statistical guidelines back to the research community.